

Patrones de Diseño

Los patrones de diseño son soluciones formales y reutilizables a problemas comunes en el desarrollo de software.

Funcionan como planos prediseñados que se pueden adaptar para resolver situaciones recurrentes en el código, evitando "reinventar la rueda" y mejorando la legibilidad y el mantenimiento de la aplicación.

Los patrones de diseño "Gang of Four" (GoF) son 23 soluciones clásicas y reutilizables para problemas comunes en la programación orientada a objetos.

Publicados en 1994 por Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides, se dividen en tres categorías principales:

1. Patrones Creacionales

Se enfocan en cómo se instancian los objetos. Su objetivo es hacer que el sistema sea independiente de cómo se crean, componen y representan sus objetos.

- **Abstract Factory:** Permite crear familias de objetos relacionados sin especificar sus clases concretas.
- **Builder:** Separa la construcción de un objeto complejo de su representación, permitiendo crear diferentes tipos y representaciones paso a paso.
- **Factory Method:** Define una interfaz para crear un objeto, pero deja que las subclasses decidan qué clase instanciar.
- **Prototype:** Especifica los tipos de objetos a crear por medio de una instancia prototipo, y crea nuevos objetos copiando este prototipo.
- **Singleton:** Garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a ella.

2. Patrones Estructurales

Abordan cómo se componen las clases y los objetos para formar estructuras más grandes y flexibles.

- **Adapter:** Permite que clases con interfaces incompatibles trabajen juntas.
- **Bridge:** Desacopla una abstracción de su implementación para que ambas puedan variar independientemente.
- **Composite:** Permite componer objetos en estructuras de árbol para representar jerarquías parte-todo.
- **Decorator:** Añade responsabilidades a un objeto de forma dinámica, proporcionando una alternativa flexible a la herencia.
- **Facade:** Proporciona una interfaz unificada que simplifica el acceso a un conjunto de interfaces en un subsistema.
- **Flyweight:** Permite compartir de manera eficiente partes comunes de estado entre múltiples objetos.
- **Proxy:** Proporciona un sustituto o marcador de posición para controlar el acceso a otro objeto.

3. Patrones de Comportamiento

Se centran en la comunicación, las interacciones y la distribución de responsabilidades entre objetos.

- **Chain of Responsibility:** Pasa solicitudes a lo largo de una cadena de posibles manejadores hasta que uno la procesa.
- **Command:** Envuelve una solicitud en un objeto, permitiendo parametrizar clientes con diferentes solicitudes y poner operaciones en cola.
- **Interpreter:** Dada una gramática, representa una interpretación de sus sentencias junto con un intérprete.
- **Iterator:** Proporciona una forma de acceder secuencialmente a los elementos de un objeto agregado sin exponer su representación subyacente.- **Mediator:** Define un objeto que encapsula cómo interactúa un grupo de objetos, promoviendo el acoplamiento débil.
- **Memento:** Captura y externaliza el estado interno de un objeto para poder restaurarlo posteriormente.
- **Observer:** Define una dependencia de uno a muchos entre objetos, de modo que cuando un objeto cambia de estado, todos sus dependientes son notificados y actualizados automáticamente.
- **State:** Permite a un objeto modificar su comportamiento cuando cambia su estado interno.
- **Strategy:** Define una familia de algoritmos, los encapsula y los hace intercambiables.
- **Template Method:** Define el esqueleto de un algoritmo en una operación, delegando algunos pasos a las subclasses.
- **Visitor:** Representa una operación a ejecutar sobre los elementos de una estructura de objeto.

[Video](#)